

使用 kubeadm 进行证书管理

使用 kubeadm 进行证书管理

由 kubeadm 生成的客户端证书在 1 年后到期。本页说明如何使用 kubeadm 管理证书续订，同时也涵盖其他与 kubeadm 证书管理相关的说明。

Kubernetes 项目建议及时升级到最新的补丁版本，并确保你正在运行受支持的 Kubernetes 次要版本。遵循这一建议有助于你确保安全。

你应该熟悉 Kubernetes 中的 PKI 证书和要求。

你应该熟悉如何将一个配置文件传递给 kubeadm 命令。

本指南将介绍如何使用 openssl 命令（用于手动证书签名），但你可以使用你喜欢的工具。

这里的一些步骤使用 sudo 来获取管理员访问权限。你可以使用任何等效的工具。

使用自定义的证书 {#custom-certificates}

默认情况下，kubeadm 会生成运行一个集群所需的全部证书。你可以通过提供你自己的证书来改变这个行为策略。

如果要这样做，你必须将证书文件放置在通过 --cert-dir 命令行参数或者 kubeadm 配置中的 certificatesDir 配置项指明的目录中。默认的值是 /etc/kubernetes/pki。

如果在运行 kubeadm init 之前存在给定的证书和私钥对，kubeadm 将不会重写它们。例如，这意味着你可以将现有的 CA 复制到 /etc/kubernetes/pki/ca.crt 和 /etc/kubernetes/pki/ca.key 中，而 kubeadm 将使用此 CA 对其余证书进行签名。

选择加密算法 {#choosing-encryption-algorithm}

kubeadm 允许你选择用于创建公钥和私钥的加密算法。这可以通过使用 kubeadm 配置中的 encryptionAlgorithm 字段来实现。

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
encryptionAlgorithm:
```

`` 可以是 RSA-2048（默认）、RSA-3072、RSA-4096 或 ECDSA-P256 之一。

选择证书有效期 {#choosing-cert-validity-period}

kubeadm 允许你选择 CA 和 leaf 证书的有效期。这可以通过使用 kubeadm 配置的 certificateValidityPeriod 和 caCertificateValidityPeriod 字段来完成：

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
certificateValidityPeriod: 8760h # 默认: 365 天 × 24 小时 = 1 年
caCertificateValidityPeriod: 87600h # 默认: 365 天 × 24 小时 * 10 = 10 年
```

字段的值遵循 Go 语言的 time.Duration 格式，支持的最长单位为 h（小时）。

外部 CA 模式 {#external-ca-mode}

只提供了 ca.crt 文件但是不提供 ca.key 文件也是可以的（这只对 CA 根证书可用，其它证书不可用）。如果所有的其它证书和 kubeconfig 文件已就绪，kubeadm 检测到满足以上条件就会激活 "外部 CA" 模式。kubeadm 将会在没有 CA 密钥文件的情况下继续执行。

否则，kubeadm 将独立运行 controller-manager，附加一个 --controllers=csrsigner 的参数，并且指明 CA 证书和密钥。

使用外部 CA 模式时，有多种方法可以准备组件证书。

手动准备组件证书 {#manual-preparation-of-component-credentials}

PKI 证书和要求包含有关如何手动准备 kubeadm 组件证书所需的所有信息。

本指南将介绍如何使用 openssl 命令（用于手动证书签名），但你可以使用你喜欢的工具。

通过签署 kubeadm 生成的 CSR 来准备证书 {#preparation-of-credentials-by-signing-csrs-generated-by-kubeadm}

kubeadm 可以生成 CSR 文件，你可以使用 openssl 和外部 CA 等工具手动签署这些文件。这些 CSR 文件将包含 kubeadm 部署的组件所需的所有证书规范。

使用 kubeadm 阶段自动准备组件证书 {#automated-preparation-of-component-credentials-by-using-kubeadm-phases}

或者，可以使用 kubeadm 阶段命令来自动化此过程。

- 登录到将作为具有外部 CA 的 kubeadm 控制平面节点的主机。
- 将外部 CA 文件 ca.crt 和 ca.key 复制到节点上的 /etc/kubernetes/pki 目录中。
- 准备一个名为 config.yaml 的临时 kubeadm 配置文件，该文件可以用在 kubeadm init 命令中。确保此文件包含可包含在证书中的集群范围或特定于主机的所有重要信息，例如 ClusterConfiguration.controlPlaneEndpoint、ClusterConfiguration.certSANs 和 InitConfiguration.APIEndpoint。
- 在同一主机上执行命令 kubeadm init stage kubeconfig all --config config.yaml 和 kubeadm init stage certs all --config config.yaml。这些操作将在 /etc/kubernetes/ 及其 pki 子目录下生成所有必需的 kubeconfig 文件和证书。
- 检查所生成的文件。删除 /etc/kubernetes/pki/ca.key，删除或移动 /etc/kubernetes/super-admin.conf 文件到安全的位置。
- 在将执行 kubeadm join 的节点上还需要删除 /etc/kubernetes/kubelet.conf，仅在将执行 kubeadm init 的第一个节点上需要此文件。
- 请注意，一些文件如 pki/sa.*、pki/front-proxy-ca.* 和 pki/etc/ca.* 在控制平面各节点上是相同的，你可以一次性生成它们并手动将其分发到将执行 kubeadm join 的节点，或者你可以使用 kubeadm init 的 --upload-certs 和 kubeadm join 的 --certificate-key 特性来执行自动分发。

在所有节点上准备好证书后，调用 kubeadm init 和 kubeadm join 命令将这些节点加入集群。kubeadm 将使用 /etc/kubernetes/ 及其 pki 子目录下现有的 kubeconfig 和证书文件。

证书过期和管理 {#check-certificate-expiration}

kubeadm 不能管理由外部 CA 签名的证书。

你可以使用 check-expiration 子命令来检查证书何时过期：

```
kubeadm certs check-expiration
```

输出类似于以下内容：

CERTIFICATE	EXPIRES	RESIDUAL TIME	CERTIFICATE AUTHORITY	EXTERNALLY MANAGED
admin.conf	Dec 30, 2020 23:36 UTC	364d		no
apiserver	Dec 30, 2020 23:36 UTC	364d	ca	no
apiserver-etcd-client	Dec 30, 2020 23:36 UTC	364d	etcd-ca	no
apiserver-kubelet-client	Dec 30, 2020 23:36 UTC	364d	ca	no
controller-manager.conf	Dec 30, 2020 23:36 UTC	364d		no
etcd-healthcheck-client	Dec 30, 2020 23:36 UTC	364d	etcd-ca	no
etcd-peer	Dec 30, 2020 23:36 UTC	364d	etcd-ca	no
etcd-server	Dec 30, 2020 23:36 UTC	364d	etcd-ca	no
front-proxy-client	Dec 30, 2020 23:36 UTC	364d	front-proxy-ca	no
scheduler.conf	Dec 30, 2020 23:36 UTC	364d		no

CERTIFICATE AUTHORITY	EXPIRES	RESIDUAL TIME	EXTERNALLY MANAGED
ca	Dec 28, 2029 23:36 UTC	9y	no
etcd-ca	Dec 28, 2029 23:36 UTC	9y	no
front-proxy-ca	Dec 28, 2029 23:36 UTC	9y	no

该命令显示 /etc/kubernetes/pki 文件夹中的客户端证书以及 kubeadm (admin.conf、controller-manager.conf 和 scheduler.conf) 使用的 kubeconfig 文件中嵌入的客户端证书的到期时间/剩余时间。

另外，kubeadm 会通知用户证书是否由外部管理；在这种情况下，用户应该小心的手动/使用其他工具来管理证书更新。

上面的列表中没有包含 kubelet.conf 配置文件，因为 kubeadm 将 kubelet 配置为自动更新证书。轮换的证书位于目录 /var/lib/kubelet/pki。要修复过期的 kubelet 客户端证书，请参阅 kubelet 客户端证书轮换失败。

在通过 kubeadm 1.17 之前的版本以 kubeadm init 创建的节点上，有一个缺陷，该缺陷使得你必须手动修改 kubelet.conf 文件的内容。kubeadm init 操作结束之后，你必须更新 kubelet.conf 文件将 client-certificate-data 和 client-key-data 改为如下所示的内容以便使用轮换后的 kubelet 客户端证书：

client-certificate: /var/lib/kubelet/pki/kubelet-client-current.pem
client-key: /var/lib/kubelet/pki/kubelet-client-current.pem

自动更新证书 {#automatic-certificate-renewal}

kubeadm 会在控制面升级的时候更新所有证书。

这个功能旨在解决最简单的用例；如果你对此类证书的更新没有特殊要求，并且定期执行 Kubernetes 版本升级（每次升级之间的间隔时间少于 1 年），则 kubeadm 将确保你的集群保持最新状态并保持合理的安全性。

如果你对证书更新有更复杂的需求，则可通过将 `--certificate-renewal=false` 传递给 `kubeadm upgrade apply` 或者 `kubeadm upgrade node`，从而选择不采用默认行为。

手动更新证书 {#manual-certificate-renewal}

你能随时通过 `kubeadm certs renew` 命令手动更新你的证书，只需带上合适的命令行选项。如果你正在运行的集群具有多副本的控制平面，则需要所有控制平面节点上执行此命令。

此命令用 CA（或者 `front-proxy-CA`）证书和存储在 `/etc/kubernetes/pki` 中的密钥执行更新。

`kubeadm certs renew` 使用现有的证书作为属性（Common Name、Organization、SAN 等）的权威来源，而不依赖于 `kubeadm-config ConfigMap`。强烈建议使它们保持同步。即便如此，Kubernetes 项目仍然建议使用的证书与 `ConfigMap` 中的关联值保持同步，以避免任何混淆的风险。

执行完此命令之后你需要重启控制面 Pod。因为动态证书重载目前还不被所有组件和证书支持，所有这项操作是必须的。静态 Pod 是被本地 kubelet 而不是 API 服务器管理，所以 `kubectl` 不能用来删除或重启他们。要重启静态 Pod 你可以临时将清单文件从 `/etc/kubernetes/manifests/` 移除并等待 20 秒（参考 `KubeletConfiguration` 结构中的 `fileCheckFrequency` 值）。如果 Pod 不在清单目录里，kubelet 将会终止它。在另一个 `fileCheckFrequency` 周期之后你可以将文件移回去，kubelet 可以完成 Pod 的重建，而组件的证书更新操作也得以完成。

`kubeadm certs renew` 可以更新任何特定的证书，或者使用子命令 `all` 更新所有的证书：

```
# 如果你运行的集群具有多副本的控制平面，则需要所有控制平面节点上执行这条命令
kubeadm certs renew all
```

复制管理员证书（可选） {#admin-certificate-copy}

使用 `kubeadm` 构建的集群通常会将 `admin.conf` 证书复制到 `$HOME/.kube/config`，参阅使用 `kubeadm` 创建集群。在这样的系统上，若要在更新 `admin.conf` 后更新 `$HOME/.kube/config` 的内容，你可以运行以下命令：

```
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

用 Kubernetes 证书 API 更新证书 {#renew-certificates-with-the-kubernetes-certificates-api}

本节提供有关如何使用 Kubernetes 证书 API 执行手动证书更新的更多详细信息。

这些是针对需要将其组织的证书基础结构集成到 kubeadm 构建的集群中的用户的高级主题。如果默认的 kubeadm 配置满足了你的需求，则应让 kubeadm 管理证书。

设置一个签名者 (Signer) {#set-up-a-signer}

Kubernetes 证书颁发机构不是开箱即用。你可以配置外部签名者，例如 cert-manager，也可以使用内置签名者。

内置签名者是 kube-controller-manager 的一部分。

要激活内置签名者，请传递 --cluster-signing-cert-file 和 --cluster-signing-key-file 参数。

如果你正在创建一个新的集群，你可以使用 kubeadm 的配置文件：

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
controllerManager:
  extraArgs:
    - name: "cluster-signing-cert-file"
      value: "/etc/kubernetes/pki/ca.crt"
    - name: "cluster-signing-key-file"
      value: "/etc/kubernetes/pki/ca.key"
```

创建证书签名请求 (CSR) {#create-certificate-signing-requests-csr}

有关使用 Kubernetes API 创建 CSR 的信息，请参见创建 CertificateSigningRequest。

通过外部 CA 更新证书 {#renew-certificates-with-external-ca}

本节提供有关如何使用外部 CA 执行手动更新证书的更多详细信息。

为了更好的与外部 CA 集成，kubeadm 还可以生成证书签名请求 (CSR)。CSR 表示向 CA 请求客户的签名证书。在 kubeadm 术语中，通常由磁盘 CA 签名的任何证书都可以作为 CSR 生成。但是，CA 不能作为 CSR 生成。

使用证书签名请求（CSR）续订 {#renewal-by-using-certificate-signing-requests-csr}

可以通过生成新的 CSR 并使用外部 CA 对其进行签名来对证书进行续约。有关使用 kubeadm 生成的 CSR 的更多详细信息，请参阅对 kubeadm 生成的证书签名请求（CSR）进行签名部分。

证书机构（CA）轮换 {#certificate-authority-rotation}

kubeadm 并不直接支持对 CA 证书的轮换或者替换。

关于手动轮换或者置换 CA 的更多信息，可参阅手动轮换 CA 证书。

启用已签名的 kubelet 服务证书 {#kubelet-serving-certs}

默认情况下，kubeadm 所部署的 kubelet 服务证书是自签名（Self-Signed）。这意味着从 metrics-server 这类外部服务发起向 kubelet 的链接时无法使用 TLS 来完成保护。

要在新的 kubeadm 集群中配置 kubelet 以使用被正确签名的服务证书，你必须向 kubeadm init 传递如下最小配置数据：

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
---
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
serverTLSBootstrap: true
```

如果你已经创建了集群，你必须通过执行下面的操作来完成适配：

- 找到 kube-system 名字空间中名为 kubelet-config 的 ConfigMap 并编辑之。在该 ConfigMap 中，kubelet 键下面有一个 KubeletConfiguration 文档作为其取值。编辑该 KubeletConfiguration 文档以设置 serverTLSBootstrap: true。
- 在每个节点上，在 /var/lib/kubelet/config.yaml 文件中添加 serverTLSBootstrap: true 字段，并使用 systemctl restart kubelet 来重启 kubelet。

字段 serverTLSBootstrap 将允许启动引导 kubelet 的服务证书，方式是从 certificates.k8s.io API 处读取。这种方式的一种局限在于这些证书的 CSR（证书签名请求）不能被 kube-controller-manager 中默认的签名组件 kubernetes.io/kubelet-serving 批准。需要用户或者第三方控制器来执行此操作。

可以使用下面的命令来查看 CSR：

```
kubectl get csr
```

NAME	AGE	SIGNERNAME	REQUESTOR	CONDITION
csr-9wvgt	112s	kubernetes.io/kubelet-serving	system:node:worker-1	Pending
csr-lz97v	1m58s	kubernetes.io/kubelet-serving	system:node:control-plane-1	Pending

你可以执行下面的操作来批准这些请求：

```
kubectl certificate approve
```

默认情况下，这些服务证书会在一年后过期。kubeadm 将 KubeletConfiguration 的 rotateCertificates 字段设置为 true；这意味着证书快要过期时，会生成一组针对服务证书的新的 CSR，而这些 CSR 也要被批准才能完成证书轮换。要进一步了解这里的细节，可参阅证书轮换文档。

如果你在寻找一种能够自动批准这些 CSR 的解决方案，建议你与你的云提供商联系，询问他们是否有 CSR 签名组件，用来以带外（out-of-band）的方式检查节点的标识符。

也可以使用第三方定制的控制器的：

- kubelet-csr-approver

除非既能够验证 CSR 中的 CommonName，也能检查请求的 IP 和域名，这类控制器还算不得安全的机制。只有完成彻底的检查，才有可能避免有恶意的、能够访问 kubelet 客户端证书的第三方为任何 IP 或域名请求服务证书。

为其他用户生成 kubeconfig 文件 {#kubeconfig-additional-users}

在集群创建过程中，kubeadm init 对 super-admin.conf 中的证书进行签名时，将其配置为 Subject: O = system:masters, CN = kubernetes-super-admin。system:masters 是一个例外的超级用户组，可以绕过鉴权层（例如 RBAC）。文件 admin.conf 也由 kubeadm 在控制平面节点上创建，此文件包含设为 Subject: O = kubeadm:cluster-admins, CN = kubernetes-admin 的证书。kubeadm:cluster-admins 是一个逻辑上属于 kubeadm 的组。如果你的集群使用 RBAC（kubeadm 的默认设置），则 kubeadm:cluster-admins 组被绑定到 cluster-admin ClusterRole。

避免共享 super-admin.conf 或 admin.conf 文件。实际上，即使是管理员等工作人员，也只为其创建最小访问权限，这种最小权限的方案适用于除例外（应急）访问之外的所有场景。

你可以使用 kubeadm kubeconfig user 命令为其他用户生成 kubeconfig 文件，这个命令支持命令行参数和 kubeadm 配置结构。以上命令会将 kubeconfig 打印到终端上，也可以使用 kubeadm kubeconfig user ... > somefile.conf 输出到一个文件中。

如下 kubeadm 可以在 --config 后加的配置文件示例：

```
# example.yaml
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
# kubernetes 将作为 kubeconfig 中集群名称
clusterName: "kubernetes"
# some-dns-address:6443 将作为集群 kubeconfig 文件中服务地址 (IP 或者 DNS 名称)
controlPlaneEndpoint: "some-dns-address:6443"
# 从本地挂载集群的 CA 秘钥和 CA 证书
certificatesDir: "/etc/kubernetes/pki"
```

确保这些设置与所需的目标集群设置相匹配。可以使用以下命令查看现有集群的设置：

```
kubectl get cm kubeadm-config -n kube-system -
o=jsonpath="{.data.ClusterConfiguration}"
```

以下示例将为在 appdevs 组的用户 johndoe 创建一个有效期为 24 小时的 kubeconfig 文件：

```
kubeadm kubeconfig user --config example.yaml --org appdevs --client-name johndoe --
validity-period 24h
```

以下示例将为管理员创建一个有效期有一周的 kubeconfig 文件：

```
kubeadm kubeconfig user --config example.yaml --client-name admin --validity-period
168h
```

签署由 kubeadm 生成的证书签名请求 (CSR) {#signing-csr}

kubeadm certs generate-csr 命令为 kubeadm 所了解并管理的所有证书生成 CSR。调用此命令将为常规证书生成 .csr / .key 文件对。对于嵌入在 kubeconfig 文件中的证书，该命令将生成一个 .csr / .conf 对，其中密钥已嵌入在 .conf 文件中。

CSR 文件包含 CA 签署证书的所有相关信息。kubeadm 对其所有证书和 CSR 使用明确定义的规约。

默认证书目录是 /etc/kubernetes/pki，而 kubeconfig 文件的默认目录是 /etc/kubernetes。这些默认值可以分别使用标志 --cert-dir 和 --kubeconfig-dir 覆盖。

要将自定义选项传递给 kubeadm certs generate-csr，可以使用 --config 标志，此标志接受 kubeadm 配置文件，与诸如 kubeadm init 这类命令相似。所有规约（例如额外的

SAN 和自定义 IP 地址）都必须存储在同一配置文件中，并通过将其作为 `--config` 传递来用于所有相关的 `kubeadm` 命令。

本指南使用默认的 Kubernetes 目录 `/etc/kubernetes`，需要超级用户权限。如果你按照本指南使用你可以写入的目录（通常这意味着使用 `--cert-dir` 和 `--kubeconfig-dir` 运行 `kubeadm`），你可以省略 `sudo` 命令。

然后，你必须将生成的文件复制到 `/etc/kubernetes` 目录下，以便 `kubeadm init` 或 `kubeadm join` 能够找到它们。

准备 CA 和服务帐户文件 {#preparing-ca-and-service-account-files}

在将执行 `kubeadm init` 的主控制平面节点上，执行以下命令：

```
sudo kubeadm init phase certs ca
sudo kubeadm init phase certs etcd-ca
sudo kubeadm init phase certs front-proxy-ca
sudo kubeadm init phase certs sa
```

这些操作将使用 `kubeadm` 所需的所有自签名 CA 文件（证书和密钥）以及服务帐户（公钥和私钥）填充控制平面节点的 `/etc/kubernetes/pki` 和 `/etc/kubernetes/pki/etcd` 目录。

如果你使用外部 CA，则你必须在带外生成相同的文件，并手动将它们复制到主控制平面节点上的 `/etc/kubernetes`。

所有 CSR 被签名后，你可以删除根 CA 密钥（`ca.key`），如外部 CA 模式部分中所述。

对于辅助控制平面节点（`kubeadm join --control-plane`），无需执行前述命令。根据你部署高可用集群的方式，你要么从主控制平面节点手动复制相同的文件，要么使用 `kubeadm init` 的 `--upload-certs` 特性实现自动化分发。

生成 CSR {#generate-csrs}

`kubeadm certs generate-csr` 命令为 `kubeadm` 所了解并管理的所有证书生成 CSR。命令完成后，你必须手动删除不需要的 `.csr`、`.conf` 或 `.key` 文件。

kubelet.conf 的注意事项 {#considerations-kubelet-conf}

本节适用于控制平面和工作节点。

如果你从控制平面节点（外部 CA 模式）上删除了 `ca.key` 文件，则该集群中的运行的 `kube-controller-manager` 将无法签署 `kubelet` 客户端证书。如果你的设置中不存在用于签署这些证书的外部方法（例如外部签名者），你可以按照本指南中的说明手动签署 `kubelet.conf.csr`。

请注意，这也意味着自动 kubelet 客户端证书轮换将被禁用。如果是这样，在证书即将到期时，你必须生成新的 kubelet.conf.csr，签署证书，将其嵌入到 kubelet.conf 中并重新启动 kubelet。

如果这不适用于你的配置，你可以跳过在辅助控制平面和工作节点（调用 kubeadm join ... 的所有节点）上处理 kubelet.conf.csr。这是因为所运行的 kube-controller-manager 将负责签署新的 kubelet 客户端证书。

你必须在主控制平面节点（你最初运行 kubeadm init 的主机）上处理 kubelet.conf.csr，这是因为 kubeadm 将该节点视为引导集群的节点，并且需要预先填充的 kubelet.conf。

控制平面节点

在主（kubeadm init）和辅助（kubeadm join --control-plane）控制平面节点上执行以下命令以生成所有 CSR 文件：

```
sudo kubeadm certs generate-csr
```

如果要使用外部 etcd，请阅读 kubeadm 使用外部 etcd 指南了解 kubeadm 和 etcd 节点上需要哪些 CSR 文件。你可以删除 /etc/kubernetes/pki/etcd 下的其他 .csr 和 .key 文件。

根据 kubelet.conf 的注意事项中的说明，你可以决定保留或删除 kubelet.conf 和 kubelet.conf.csr 文件。

工作节点

根据 kubelet.conf 的注意事项中的解释，可以选择执行：

```
sudo kubeadm certs generate-csr
```

并仅保留 kubelet.conf 和 kubelet.conf.csr 文件，或者完全跳过工作节点的步骤。

签署所有证书的 CSR {#signing-csrs-for-all-certificates}

如果你使用外部 CA 并且已经拥有 openssl 的 CA 序列号文件（.srl），你可以将此类文件复制到将处理 CSR 的 kubeadm 节点。要复制的 .srl 文件有 /etc/kubernetes/pki/ca.srl、/etc/kubernetes/pki/front-proxy-ca.srl 和 /etc/kubernetes/pki/etcd/ca.srl。然后将文件移动到将处理 CSR 文件的新节点。

如果节点上的 CA 缺少 .srl 文件，下面的脚本将生成一个具有随机起始序列号的新 SRL 文件。

要了解有关 .srl 文件的更多信息，请参阅 openssl 关于 --CAserial 标志的文档。

对具有 CSR 文件的所有节点重复此步骤。

在 /etc/kubernetes 目录中编写以下脚本，进入该目录并执行该脚本。该脚本将为 /etc/kubernetes 目录下存在的所有 CSR 文件生成证书。

```
#!/bin/bash
```

```
# 设置证书过期时间（以天为单位）
```

```
DAYS=365
```

```
# 处理除 front-proxy 和 etcd 之外的所有 CSR 文件
```

```
find ./ -name "*.csr" | grep -v "pki/etcd" | grep -v "front-proxy" | while read -r FILE;
do
    echo "* Processing ${FILE} ..."
    FILE=${FILE%.*} # 修剪扩展名
    if [ -f "./pki/ca.srl" ]; then
        SERIAL_FLAG="-CAserial ./pki/ca.srl"
    else
        SERIAL_FLAG="-CAcreateserial"
    fi
    openssl x509 -req -days "${DAYS}" -CA ./pki/ca.crt -CAkey ./pki/ca.key ${SERIAL_FLAG} \
        -in "${FILE}.csr" -out "${FILE}.crt"
    sleep 2
done
```

```
# 处理所有 etcd CSR
```

```
find ./pki/etcd -name "*.csr" | while read -r FILE;
do
    echo "* Processing ${FILE} ..."
    FILE=${FILE%.*} # 修剪扩展名
    if [ -f "./pki/etcd/ca.srl" ]; then
        SERIAL_FLAG=-CAserial ./pki/etcd/ca.srl
    else
        SERIAL_FLAG=-CAcreateserial
    fi
    openssl x509 -req -days "${DAYS}" -CA ./pki/etcd/ca.crt -CAkey ./pki/etcd/ca.key $
{SERIAL_FLAG} \
        -in "${FILE}.csr" -out "${FILE}.crt"
done
```

处理前端代理 CSR

```
echo "* Processing ./pki/front-proxy-client.csr ..."
openssl x509 -req -days "${DAYS}" -CA ./pki/front-proxy-ca.crt -CAkey ./pki/front-proxy-ca.key -CAcreateserial \
-in ./pki/front-proxy-client.csr -out ./pki/front-proxy-client.crt
```

在 kubeconfig 文件中嵌入证书 {#embedding-certificates-in-kubeconfig-files}

对具有 CSR 文件的所有节点重复此步骤。

在 /etc/kubernetes 目录中编写以下脚本，进入该目录并执行脚本。此脚本将基于上一步从 CSR 中得到为 kubeconfig 文件签名的 .crt 文件，并将它们嵌入到 kubeconfig 文件中。

#!/bin/bash

```
CLUSTER=kubernetes
find ./ -name "*.conf" | while read -r FILE;
do
    echo "* Processing ${FILE} ..."
    KUBECONFIG="${FILE}" kubectl config set-cluster "${CLUSTER}" --certificate-authority
    ./pki/ca.crt --embed-certs
    USER=$(KUBECONFIG="${FILE}" kubectl config view -o jsonpath='{.users[0].name}')
    KUBECONFIG="${FILE}" kubectl config set-credentials "${USER}" --client-certificate "${FILE}.crt" --embed-certs
done
```

执行清理 {#post-csr-cleanup}

在具有 CSR 文件的所有节点上执行此步骤。

在 /etc/kubernetes 目录中编写以下脚本，进入该目录并执行脚本。

#!/bin/bash

清理 CSR 文件

```
rm -f ./*.csr ./pki/*.csr ./pki/etcd/*.csr # 清理所有 CSR 文件
```

清理已嵌入 kubeconfig 文件中的 CRT 文件

```
rm -f ./*.crt
```

(可选) 将 .srl 文件移动到下一个要处理的节点。

或者，如果使用外部 CA，请删除 `/etc/kubernetes/pki/ca.key` 文件，如外部 CA 节点部分中所述。

kubeadm 节点初始化 {#kubeadm-node-initialization}

一旦 CSR 文件被签名并且所需的证书在要用作节点的主机上就位，你就可以使用命令 `kubeadm init` 和 `kubeadm join` 使用这些节点创建 Kubernetes 集群。在 `init` 和 `join` 期间，`kubeadm` 使用在主机本地文件系统的 `/etc/kubernetes` 目录中找到的现有证书、加密密钥和 `kubeconfig` 文件。